Генерация кадров и апскейлинг

Область

Документ описывает политику рендера для:

- FSR1 spatial-апскейлинга;
- FSR2 temporal-апскейлинга;
- DLSS temporal-апскейлинга на NVIDIA;
- FSR3 FG;
- DLSS FG на NVIDIA;
- собственной генерации кадров.

XeSS-апскейлинг и XeSS FG в продукт не входят. MetalFX остаётся

зарезервированным значением в схеме без адаптера.

Также описано взаимодействие с HUD, render scale, OIT, запеканием статических

слоёв, alpha cutout и батчингом draw calls.

Все перечисленные технологии по умолчанию выключены. Проект должен явно включить

нужную технологию. Кроме того, до появления опции в интерфейсе должен пройти

проверка возможностей устройства.

FSR1, FSR2, FSR3 FG, DLSS/DLSS FG и собственная генерация имеют контракты в

движке. Vendor-backend’ы динамически загружают runtime DLL; при отсутствии

runtime или неподдерживаемом железе выполняется fallback без краша.

Термины

Spatial-апскейлинг восстанавливает изображение из кадра меньшего разрешения без

временной истории. К этой группе относятся FSR1 и bilinear-фильтрация.

Temporal-апскейлинг использует цвет, глубину, motion vectors и предыдущие кадры.

К этой группе относятся FSR2 и DLSS.

Генерация кадров вставляет дополнительный кадр между двумя отрендеренными

кадрами. Она не заменяет апскейлинг и работает после того, как сцена уже

приведена к разрешению экрана.

HUD — это экранный интерфейс, текст, меню, курсор, субтитры и слои доступности. Для генерации кадров HUD не часть игровой сцены.

Порядок рендера

Поддерживаемый порядок:

1. игровая сцена рендерится в scene target;
2. сцена при необходимости масштабируется до разрешения экрана;
3. при необходимости генерируются промежуточные кадры;
4. HUD рендерится в отдельный target в нативном разрешении;
5. сгенерированная сцена смешивается с HUD;
6. результат выводится на экран.

HUD не должен попадать в motion vectors, глубину, reactive mask или историю генерации. HUD накладывается после апскейлинга и после генерации кадров.

Минимальные данные конвейера:

- цвет сцены;
- глубина;
- motion vectors камеры;
- motion vectors спрайтов;
- reactive mask для прозрачности, частиц и OIT;
- предыдущая камера;
- признак валидности истории;
- размеры render и output.

Собственная генерация кадров

Собственный генератор работает после выбранного апскейлера и не зависит от производителя GPU.

Вход:

- цвет сцены в разрешении экрана;
- глубина в разрешении рендера;
- motion vectors в разрешении рендера;
- reactive mask;
- смещение камеры;
- jitter и состояние истории;
- тайминг кадра.

Выход — кадр сцены в разрешении экрана. HUD не выход генератора.

Этапы:

7. проверка истории и motion vectors;
8. репроекция предыдущей сцены;
9. определение disocclusion по глубине;
10. применение reactive mask к нестабильным пикселям;
11. интерполяция сцены;
12. смешивание с текущим реальным кадром;
13. пометка кадра как сгенерированного или реального.

Генератор обязан поддерживать fail-safe. Если уверенность в движении низкая,

сцена сброшена, тайминг нестабилен или изменился размер рендера, выдаётся

реальный кадр вместо интерполированного.

Настройки

Политика проекта хранится в `project.toml`:

```
[render]
render_scale = 1.0
upscale_policy = "auto"
frame_generation_policy = "auto"
allow_temporal_upscale = true
allow_frame_generation = true
```

Настройки игрока хранятся отдельно:

```
[graphics]
render_scale = 1.0
upscale_policy = "auto"
frame_generation_policy = "auto"
```

Игрок выбирает политику, а не вендора:

- Auto: DLSS на NVIDIA, иначе FSR2, иначе FSR1; DLSS FG на NVIDIA, иначе FSR3

FG, иначе custom;

- Universal: FSR2/FSR1 и FSR3 FG/custom;
- Off: bilinear без генерации.

Vendor-значения остаются в project/user-файлах для кросс-платформенности.

Интерфейс фильтрует недоступные значения по capabilities.

Проверка возможностей

Возможности сообщает RHI backend:

```
struct UpscalerCaps
{
```

```
    bool nearest;
    bool bilinear;
    bool bicubic;
    bool fsr1;
    bool fsr2;
    bool dlss;
    bool motionVectors;
    bool frameGeneration;
};
```

Backend доступен только при выполнении всех условий:

- FSR1: `fsr1`;
- FSR2: `fsr2 && motionVectors`;
- DLSS: Vulkan + Windows + NVIDIA + NGX runtime + motion vectors;
- FSR3 FG: FidelityFX runtime + motion vectors;
- DLSS FG: NVIDIA + DLSS-G feature + motion vectors;
- собственная генерация: temporal resolve + motion vectors + HUD separation.

Опциональные флаги сборки:

```
FBY_WITH_DLSS=OFF
FBY_WITH_FIDELITYFX=OFF
```

FSR1/FSR2/custom FG — core-пути с runtime-gating. Если vendor backend не собран или его runtime отсутствует, capability равен false, настройки уходят в fallback. XeSS удалён, MetalFX зарезервирован без адаптера.

Опциональные vendor SDK

SDK DLSS и FidelityFX не коммитятся в репозиторий движка. Они большие, лицензируются вендором и опциональны; checkout без них всё равно собирает переносимые пути FSR1/FSR2/custom. Передайте `-WithVendorSdks -AcceptVendorLicenses` в Windows `Setup.bat` или `--with-vendor-sdks --accept-vendor-licenses` в Linux/macOS `Setup`, чтобы склонировать SDK с GitHub в `third_party/`. Разработчик также может установить SDK локально и указать путь CMake:

```
# NVIDIA DLSS, только Windows + Vulkan + NVIDIA
cmake -S . -B build -DFBY_WITH_DLSS=ON -DFBY_DLSS_ROOT="C:/SDK/NVIDIA-DLSS"

# AMD FidelityFX, адаптер FSR2/FSR3
cmake -S . -B build -DFBY_WITH_FIDELITYFX=ON \
      -DFBY_FIDELITYFX_ROOT="C:/SDK/FidelityFX/Kits/FidelityFX"
```

Ожидаемая раскладка:

- DLSS: `include/nvsdk_ngx.h` и runtime DLL NGX;
- FidelityFX: `api/include/ffx_api.h`, `upscalers/`, `framegeneration/`, `backend/vk/` и `signedbin/amd_fidelityfx_upscaler_vk.dll`.

`FBY_DLSS_ROOT` и `FBY_FIDELITYFX_ROOT` — cache-пути, поэтому SDK можно держать вне репозитория. Отсутствующий SDK даёт предупреждение configure: vendor backend отключается, configure завершается, движок и редактор собираются, а выбор в рантайме уходит в fallback. Передайте `-DFBY_REQUIRE_VENDOR_SDKS=ON`, если CI должен падать на отсутствующем SDK. DLL FidelityFX копируется рядом с executable; DLSS загружает NGX runtime динамически из поискового пути драйвера/SDK. Не копируйте SDK в исходники; стандартные `third_party/dlss` и `third_party/fidelityfx` игнорируются и служат только локальными корнями SDK.

Взаимное исключение

Одновременно допускается один апскейлер и не более одного генератора кадров.

Допустимые комбинации:

- native + собственная генерация;
- FSR1 + собственная генерация;
- FSR2 + собственная генерация после тестов;
- DLSS + собственная генерация после тестов;
- native + vendor-генерация.

Недопустимые комбинации:

- FSR2 и DLSS одновременно;
- два генератора кадров одновременно;
- DLSS Frame Generation и собственная генерация;
- FSR3 Frame Generation и собственная генерация;
- HUD внутри входа генерации.

Движок разрешает недопустимые комбинации до рендера. Он не запускает два backend одновременно. Политика проекта имеет приоритет над выбором игрока. Проверка возможностей имеет приоритет над обоими.

Матрица совместимости

Апскейлер	Собственная генерация	Vendor-генерация
Native	Поддерживается	Только vendor
FSR1	Рекомендуется	Только vendor
FSR2	После тестов	Не с собственной
DLSS	После тестов	Не с собственной
Bilinear/Bicubic	Поддерживается	Только vendor

FSR1 вместе с собственной генерацией — предпочтительный переносимый вариант, потому что у FSR1 нет собственной временной истории. FSR2 или DLSS вместе с собственной генерацией создают два temporal-этапа и требуют проверки на ghosting.

Правила HUD

HUD рендерится в нативном разрешении. Допускается отдельный HUD render scale, но он не зависит от render scale игры.

Режимы HUD target:

- `Direct`: UI рисуется прямо в swapchain. Самый быстрый путь, offscreen HUD target не выделяется.
- `Offscreen`: UI рисуется в HUD target и композитится после генерации.
- `Auto`: Direct без генерации, Offscreen с генерацией.

`hud.render_scale`, `hud.upscale` и `hud.visible` относятся к offscreen-пути.

Режим target доступен в настройках игры; проект может его ограничить.

`Direct` при активной генерации отключает генерацию, а не помещает текст в temporal-историю.

HUD не должен передаваться в FSR2, DLSS, vendor-генерацию или собственную генерацию. Качество текста сохраняется только вне temporal-конвейера.

Разрешение коллизий

Коллизии разрешаются в порядке:

14. политика проекта;
15. наличие backend в сборке;
16. возможности устройства;

17. выбор игрока;
18. fallback.

Правила:

- проект, запретивший temporal, скрывает FSR2 и DLSS;
- устройство без motion vectors отключает все temporal-пути;
- устройство без backend генерации отключает генератор;
- vendor-генерация и собственная генерация не работают вместе;
- смена апскейлера сбрасывает историю temporal и генерации;
- смена render scale или размера экрана сбрасывает обе истории;
- смена режима HUD не влияет на историю сцены;
- OIT, частицы и прозрачность обязаны выставлять reactive mask;
- изменения HUD не должны сбрасывать историю сцены.

Если выбранный backend исчез после пересоздания устройства, движок переходит на native-рендер и сообщает причину в диагностике.

Артефакты

Артефакт	Причина	Мера
Блочность	Низкий render scale и nearest	Bilinear, bicubic, FSR1, выше render scale
Мыло	Bilinear или слабый spatial upscale	FSR1, bicubic, выше render scale
Ghosting	Неверные motion vectors или старая история	Сброс истории, проверка векторов, reactive mask
Мазня	Прозрачность, частицы или OIT в истории	Reactive mask, отключение генерации для эффекта
Дрожание краёв	Нет jitter или неверная глубина	Корректные jitter, depth и motion vectors
Ghosting текста	HUD внутри генерации	HUD нативно и после генерации
Задержка ввода	Поздний вывод сгенерированных кадров	Frame pacing, latency budget, отключение при низком FPS
Двойной ghosting	FSR2/DLSS и собственная генерация	Тесты или FSR1 с генерацией
Дрожание pixel art	Субпиксельная генерация на целых пикселях	Отключить генерацию для pixel-art пресета

Лучшие варианты по типу игры

Pixel art с целым масштабом: nearest и без генерации. Render scale 1.0, если GPU позволяет.

Painted 2D с движением камеры: FSR1 — безопасный переносимый выбор. Собственная генерация допустима при корректных motion vectors и отдельном HUD.

Быстрый экшен с прозрачностью: FSR1 и собственная генерация только после проверки reactive mask. При мазне частиц генерацию отключить.

Игры с большим HUD: HUD только нативно. Не включать генерацию, если HUD нельзя отделить от сцены.

Статичные фоны и медленная камера: static bake и FSR1 дают наибольший выигрыш при минимуме temporal-артефактов.

Скриншоты и бенчмарки: HUD `hidden`, генерация выключена, native-рендер для воспроизводимости.

Другие технологии движка

Render scale уменьшает внутреннее разрешение сцены и не влияет на HUD. Снижает fill rate, но требует апскейлера для вывода в разрешение экрана.

OIT — order-independent transparency с accumulation и revealage target. Пиксели OIT являются reactive; генерация не должна считать их стабильной историей без маски.

Static layer bake переиспользует отрендеренную сцену, пока камера, сцена и свет не изменились. Кэш сбрасывается при движении камеры, изменении света, сцены, post-processing и SDF.

Alpha cutout переводит opaque-арт с бинарной alpha в depth-tested путь. Он безопасен для front-to-back только если cooked-метаданные помечают слой как opaque или cutout. Blended-арт остаётся на обычном пути.

Single-indirect opaque rendering объединяет совместимые opaque-батчи в один indirect draw. Это не меняет стоимость пикселей и не заменяет апскейлинг.

Проверка

- все backend выключены по умолчанию;
- caps скрывают неподдерживаемые опции;
- политика проекта приоритетнее выбора игрока;
- только один апскейлер;
- только один генератор;

- HUD нативно и вне temporal-истории;
- сброс при смене размера, масштаба, сцены и камеры;
- reactive mask для OIT и частиц;
- fallback на реальный кадр при низкой уверенности;
- pixel-art пресет отключает генерацию;
- диагностика сообщает активный backend и причину fallback.